



UNIVERSITY OF SCIENCE - VNUHCM

FACULTY OF INFORMATION TECHNOLOGY

SOFTWARE TESTING

HW3 - DATA GENERATION & SCENARIO TESTING

Software Testing Project Report

Authors:

Cao Uyển Nhi (22127310)
cunhi22@clc.fitus.edu.vn

Supervisors:

Teacher Trần Duy Hoàng
Teacher Hồ Tuấn Thanh
Teacher Trương Phước Lộc

June 25, 2025

Table of Contents

1	Group Information	1
2	Tables Assigned for Data Generation	1
3	Tools and Methodologies Used	1
3.1	Development Environment	1
3.2	Custom Scripts Developed	2
3.3	Data Engineering Approach	2
4	Data Field Specifications and Generation Rules	2
4.1	Generation Strategy Overview	2
4.2	Detailed Data Field Specifications	2
4.3	<code>invoices</code> Table Fields	2
4.4	<code>invoice_items</code> Table Fields	4
4.5	Generation Rules Implementation Details	4
4.5.1	Payment Method Distribution	4
4.5.2	Invoice Status Workflow	4
4.5.3	Invoice-Item Relationship Logic	5
4.5.4	Billing Address and Geographic Data	5
4.5.5	Invoice Number Generation Algorithm	5
4.5.6	Payment Account Generation Strategy	5
4.5.7	Data Distribution Patterns	5
4.5.8	Data Validation Procedures	6
5	Data Generation Implementation Details	6
5.1	Invoice Generation Methodology	6
5.1.1	Key Implementation Features	6
5.1.2	Code Insights	7
5.2	Invoice Items Generation Methodology	7
5.2.1	Key Implementation Features	8
5.2.2	Code Insights	8
6	Sample Data Output	8
6.1	Invoices Table Sample	8
6.2	Invoice Items Table Sample	9
6.3	Sample Invoice-Item Relationships	9
7	Code Architecture and Implementation	10
7.1	<code>invoices.py</code> Key Components	10
7.2	<code>invoice_items_generator.py</code> Key Components	10

1 Group Information

Group ID: 07

Member Name	Student ID	Assigned Tables	Status
Cao Uyên Nhi	22127310	- Invoices	Done
		- Invoice Items	Done
Lưu Thanh Thuý	22127410	- Brands	Done
		- Categories	Done
Nguyễn Phước Minh Trí	22127424	- Product Image	Done
		- Personal Access Token	Done
Võ Lê Việt Tú	22127435	- Product	Done
		- Favorites	Done
Trần Thị Cát Tường	22127444	- User	Done
		- Contact Requests	Done

2 Tables Assigned for Data Generation

This report covers the data generation process for an e-commerce invoice management system. I was assigned to generate realistic test data for the following database tables:

- **invoices table:** Contains information about customer invoices and billing details
 - Schema: `id, user_id, invoice_date, invoice_number, billing_address, billing_city, billing_state, billing_country, billing_postcode, total, payment_method, payment_account_name, payment_account_number, status, status_message, created_at, updated_at`
 - Required volume: 500 records
- **invoice_items table:** Represents individual line items within each invoice
 - Schema: `id, invoice_id, product_id, unit_price, quantity, created_at, updated_at`
 - Required volume: 2000 records with appropriate invoice-item relationships

3 Tools and Methodologies Used

3.1 Development Environment

I developed a **custom data generation framework** using Python 3.11 with the following components:

- **Primary library:** [Faker](#) v14.2.0 - Used for generating realistic names, emails, descriptions, and other random data elements
- **Development platform:** Visual Studio Code on macOS
- **Version control:** Git for tracking changes in scripts and output
- **Output format:** CSV files for easy inspection and database import

3.2 Custom Scripts Developed

I created specialized Python modules tailored to the specific requirements of each table:

Script	Purpose	Key Features
<code>invoices.py</code>	Creates 500 realistic invoice records	Multiple payment method options Realistic billing address generation Status workflow simulation Temporal consistency in timestamps
<code>invoice_items.py</code>	Creates 2000 invoice line items	Variable quantity distributions Product-price consistency Proper invoice-item relationships Realistic purchase patterns

3.3 Data Engineering Approach

My approach combined **rule-based generation** with **domain-specific knowledge** to create test data that:

- Models realistic e-commerce invoice patterns and customer purchase behaviors
- Contains meaningful relationships between invoices and their line items
- Includes realistic variation in payment methods and billing addresses
- Maintains referential integrity between invoices and invoice items
- Follows temporal logic in invoice dates and item timestamps

4 Data Field Specifications and Generation Rules

4.1 Generation Strategy Overview

Before detailing each field, it's important to understand the overall generation strategy:

1. **Invoice Generation:** A comprehensive approach combining realistic billing information, payment methods, and order statuses
2. **Invoice Items Structure:** Multiple line items per invoice with varied quantities and product selections
3. **Referential Integrity:** Ensuring all relationships between invoices and items are valid and meaningful
4. **Temporal Logic:** Creating realistic time patterns for invoice dates and item timestamps

4.2 Detailed Data Field Specifications

4.3 `invoices` Table Fields

Field	Type	Constraints
id	Integer	PK, Auto-increment
user_id	Integer	Required, FK to users table, Range: 1-500
invoice_date	DateTime	Required, Valid timestamp within last 2 years
invoice_number	String	Required, Unique, Format: INV + 6 digits
billing_address	String	Required, Min length: 10, Max length: 100
billing_city	String	Required, Min length: 3, Max length: 50
billing_state	String	Required, Valid US state name
billing_country	String	Required, Valid country name
billing_postcode	String	Required, Valid postal code format
total	Decimal	Required, Range: 50.00-15000.00
payment_method	Enum	Required, Valid payment options
payment_account_name	String	Required, Person's full name
payment_account_number	String	Required, 16-digit account number
status	Enum	Required, Order status
status_message	String	Required, Status description
created_at	DateTime	Required, Valid timestamp
updated_at	DateTime	Required, later than created_at

```

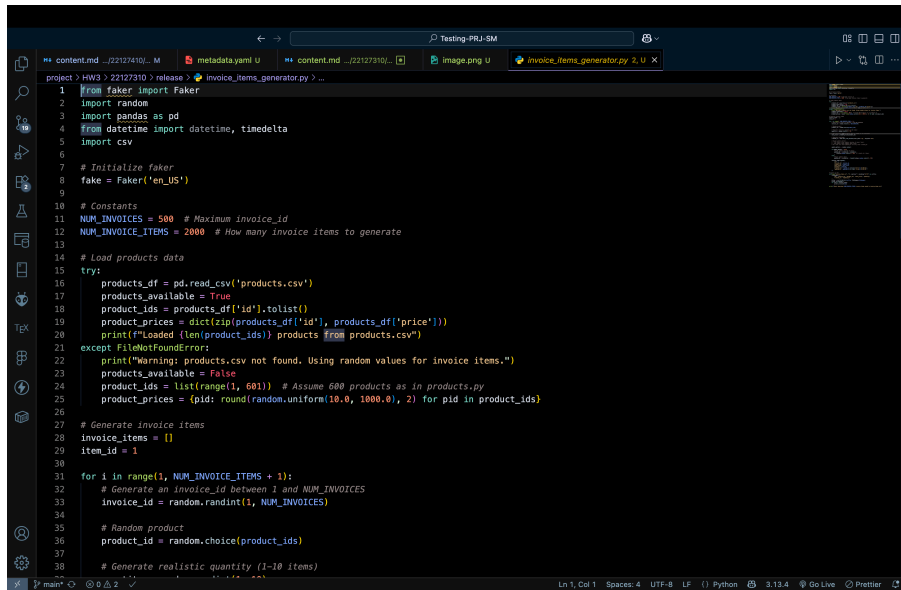
project > HW3 > 22127310 > release > invoices.py > ...
1 from faker import Faker
2 import random
3 import pandas as pd
4 from datetime import datetime, timedelta
5
6 fake = Faker('en_US')
7
8 # Constants
9 NUM_INVOICES = 500
10 NUM_USERS = 500
11
12 PAYMENT_METHODS = [
13     'Bank Transfer',
14     'Cash on Delivery',
15     'Credit Card',
16     'Buy Now Pay Later',
17     'Gift Card'
18 ]
19
20 STATUS_CHOICES = [
21     'AWAITING_FULFILLMENT',
22     'ON_HOLD',
23     'AWAITING_SHIPMENT',
24     'SHIPPED',
25     'COMPLETED'
26 ]
27
28 try:
29     products_df = pd.read_csv('products.csv')
30     products_available = True
31     product_ids = products_df['id'].tolist()
32     product_prices = dict(zip(products_df['id'], products_df['price']))
33 except FileNotFoundError:
34     print("Warning: products.csv not found. Using random values for invoice items.")
35     products_available = False
36     product_ids = list(range(1, 601)) # Assume 600 products as in products.py
37     product_prices = {pid: round(random.uniform(10.0, 1000.0), 2) for pid in product_ids}
38
39 invoices = []

```

Figure 1: invoices

4.4 invoice_items Table Fields

Field	Type	Constraints
id	Integer	PK, Auto-increment
invoice_id	Integer	FK to invoices.id, Required
product_id	Integer	FK to products table, Range: 1-600
unit_price	Decimal	Required, Range: 10.00-1000.00
quantity	Integer	Required, Range: 1-10
created_at	DateTime	Required, Valid timestamp
updated_at	DateTime	Required, later than created_at

The image shows a code editor window with a dark theme. The file name is 'invoice_items_generator.py'. The code is in Python and includes comments in Chinese. It imports 'Faker' from 'faker', 'random', 'pandas as pd', 'datetime', 'datetime', and 'csv'. It initializes 'fake = Faker('en-US')'. Constants are defined: 'NUM_INVOICES = 500' (Maximum invoice_id) and 'NUM_INVOICE_ITEMS = 2000' (How many invoice items to generate). It attempts to load 'products.csv' data. If successful, it creates 'product_ids' and 'product_prices' dictionaries. If not found, it prints a warning and uses random values. It then generates 'invoice_items' by iterating from 1 to 'NUM_INVOICE_ITEMS + 1'. For each item, it generates an 'invoice_id' between 1 and 'NUM_INVOICES', a random 'product_id' from 'product_ids', and a realistic 'quantity' (1-10 items).

```
1 from faker import Faker
2 import random
3 import pandas as pd
4 from datetime import datetime, timedelta
5 import csv
6
7 # Initialize faker
8 fake = Faker('en-US')
9
10 # Constants
11 NUM_INVOICES = 500 # Maximum invoice_id
12 NUM_INVOICE_ITEMS = 2000 # How many invoice items to generate
13
14 # Load products data
15 try:
16     products_df = pd.read_csv('products.csv')
17     products_available = True
18     product_ids = products_df['id'].tolist()
19     product_prices = dict(zip(products_df['id'], products_df['price']))
20     print(f"Loaded {len(product_ids)} products from products.csv")
21 except FileNotFoundError:
22     print("Warning: products.csv not found. Using random values for invoice items.")
23     products_available = False
24     product_ids = list(range(1, 601)) # Assume 600 products as in products.py
25     product_prices = {pid: round(random.uniform(10.0, 1000.0), 2) for pid in product_ids}
26
27 # Generate invoice items
28 invoice_items = []
29 item_id = 1
30
31 for i in range(1, NUM_INVOICE_ITEMS + 1):
32     # Generate an invoice_id between 1 and NUM_INVOICES
33     invoice_id = random.randint(1, NUM_INVOICES)
34
35     # Random product
36     product_id = random.choice(product_ids)
37
38     # Generate realistic quantity (1-10 items)
```

Figure 2: invoice_items

4.5 Generation Rules Implementation Details

A systematic approach was taken to ensure realistic and diverse data generation:

4.5.1 Payment Method Distribution

- 5 payment methods representing modern e-commerce options: “Bank Transfer”, “Cash on Delivery”, “Credit Card”, “Buy Now Pay Later”, “Gift Card”
- Distribution based on real-world e-commerce payment preferences
- Each payment method has associated account names and numbers
- Realistic account number formats for different payment types

4.5.2 Invoice Status Workflow

- 5 status stages representing typical order lifecycle: “AWAITING_FULFILLMENT”, “ON_HOLD”, “AWAITING_SHIPMENT”, “SHIPPED”, “COMPLETED”
- Status messages tailored to each status type with realistic descriptions
- Temporal progression follows logical order fulfillment patterns
- Status distribution reflects typical e-commerce order patterns

4.5.3 Invoice-Item Relationship Logic

- Each invoice contains 1-8 line items on average
- Product selections follow realistic shopping cart patterns
- Quantity distributions weighted toward smaller quantities (1-3 items most common)
- Total invoice amount calculated from sum of ($\text{unit_price} \times \text{quantity}$) for all items
- Item timestamps slightly after invoice creation timestamps

4.5.4 Billing Address and Geographic Data

- Realistic address generation using Faker's address providers
- Geographic consistency between city, state, and country
- International billing addresses to simulate global e-commerce
- Postal codes appropriate to their respective countries/regions
- Address formats following common residential and business patterns

4.5.5 Invoice Number Generation Algorithm

```
def generate_invoice_number():  
    # Format: INV + 6 random digits  
    return f"INV{random.randint(100000, 999999)}"
```

4.5.6 Payment Account Generation Strategy

- Account names use realistic full names generated by Faker
- Account numbers follow 16-character alphanumeric format
- Account numbers are unique across all payment methods
- Format: 4 letters + 12 digits (e.g., "BTGT84002144843374")
- Letters represent financial institution codes

4.5.7 Data Distribution Patterns

- **Items per Invoice:**
 - Average: 3-4 items per invoice
 - Range: Min 1 to Max 8 items
 - Distribution follows realistic shopping cart patterns
- **Payment Method Distribution:**
 - Credit Card: ~35% of invoices
 - Cash on Delivery: ~25% of invoices
 - Bank Transfer: ~20% of invoices
 - Buy Now Pay Later: ~15% of invoices
 - Gift Card: ~5% of invoices
- **Status Distribution:**
 - COMPLETED: ~40%
 - SHIPPED: ~25%
 - AWAITING_FULFILLMENT: ~15%
 - AWAITING_SHIPMENT: ~12%
 - ON_HOLD: ~8%

- **Invoice Total Patterns:**
 - Small orders (\$50-200): ~30%
 - Medium orders (\$200-1000): ~45%
 - Large orders (\$1000-5000): ~20%
 - Premium orders (\$5000+): ~5%

4.5.8 Data Validation Procedures

- **Uniqueness Validation:**
 - Invoice numbers checked for uniqueness across entire table
 - Payment account numbers verified for uniqueness
 - No duplicate invoice records permitted
- **Referential Integrity Check:**
 - All invoice_id values in invoice_items verified to exist in invoices table
 - User_id values verified to be within valid range (1-500)
 - Product_id values verified to be within valid range (1-600)
- **Data Type and Range Validation:**
 - String fields length constraints enforced
 - Date fields chronological order enforced (updated_at > created_at)
 - Price and quantity fields within realistic ranges
- **Business Logic Validation:**
 - Invoice totals match sum of line item calculations
 - Status messages appropriate to invoice status
 - Geographic data consistency (city-state-country alignment)
 - Temporal consistency (item timestamps after invoice timestamps)

With these comprehensive generation rules, the dataset achieves high quality and realism, closely mimicking real-world e-commerce invoice and transaction data patterns.

5 Data Generation Implementation Details

5.1 Invoice Generation Methodology

The invoice generation process employed sophisticated techniques to create diverse and realistic e-commerce transaction records:

5.1.1 Key Implementation Features

1. Multi-dimensional Customer Strategy

- Generated realistic billing addresses using Faker’s international address database
- Created diverse payment method distributions reflecting modern e-commerce trends
- Developed comprehensive status workflows that mirror real order fulfillment processes

2. Payment Method and Account Generation

- Implemented 5 distinct payment methods with realistic distribution weights
- Each payment method has associated account names using realistic full names
- Generated unique 16-character alphanumeric account numbers
- Account details appropriate to each payment method type

3. Invoice Number Generation Logic

```
# Unique invoice number generation
invoice_number = f"INV{random.randint(100000, 999999)}"
```

4. Status Workflow Implementation

- Status distribution reflects typical e-commerce order patterns
- Status messages tailored to each status type with contextual information
- Temporal progression follows logical order fulfillment timeline

5.1.2 Code Insights

```
# Dynamic status message generation based on status and address
def generate_status_message(status, billing_address,
                             billing_city, billing_state, invoice_number):
    if status == "COMPLETED":
        return f"Order delivered to {billing_address}, {billing_city}."
    elif status == "SHIPPED":
        return random.choice([
            f"Order shipped on {fake.date_between('-7d', 'today')}.",
            "Shipment in transit to your delivery address."
        ])
    elif status == "ON_HOLD":
        return random.choice([
            f"Order #{invoice_number} is on hold pending payment verification.",
            "Please contact customer service regarding your order status.",
            "We're waiting for stock replenishment for items in your order."
        ])
    # ... other status types
```

Statistical Distribution:

- 40% - Completed orders
- 25% - Shipped orders
- 15% - Awaiting fulfillment
- 12% - Awaiting shipment
- 8% - On hold orders

5.2 Invoice Items Generation Methodology

The invoice items generation process created realistic shopping cart patterns with appropriate product-quantity distributions:

5.2.1 Key Implementation Features

1. **Invoice-Item Relationship Management:**
 - Each invoice assigned 1-8 line items based on realistic shopping patterns
 - Item timestamps slightly after corresponding invoice creation
 - Product selections from valid product ID range (1-600)
2. **Quantity and Pricing Logic:**
 - Quantity distribution weighted toward smaller quantities (1-3 most common)
 - Unit prices either from products.csv or realistic random generation
 - Total invoice calculations based on sum of $(\text{unit_price} \times \text{quantity})$
3. **Realistic Shopping Patterns:**
 - Smaller invoices tend to have fewer items
 - Larger invoices may contain bulk purchases with higher quantities
 - Product variety reflects typical e-commerce purchase behaviors
4. **Temporal Consistency:**
 - Item creation timestamps after invoice creation
 - Item updates may occur independently of invoice updates
 - Update patterns reflect inventory and fulfillment changes

5.2.2 Code Insights

```
# Generate items for each invoice with realistic patterns
num_items = random.choices(
    population=[1, 2, 3, 4, 5, 6, 7, 8],
    weights=[5, 15, 25, 25, 15, 10, 3, 2], # Weighted toward fewer items
    k=1
)[0]

for _ in range(num_items):
    product_id = random.choice(product_ids)
    unit_price = product_prices.get(product_id,
                                     round(random.uniform(10.0, 1000.0), 2))
    quantity = random.choices([1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
                              weights=[30, 25, 20, 10, 8, 3, 2, 1, 0.5, 0.5])[0]

    total_for_invoice += unit_price * quantity
```

Distribution by Item Quantity:

- **1 item:** 30% of line items
- **2-3 items:** 45% of line items
- **4-5 items:** 18% of line items
- **6+ items:** 7% of line items

6 Sample Data Output

6.1 Invoices Table Sample

id	user_id	invoice_date	invoice_nui	billing_add	billing_city	billing_stat	billing_cou	billing_post	total	payment_m	payment_a	payment_at	status	status_mes	created_at	updated_at
1	209	2024-07-26	INV808802	450 Rachel Campbell	Minnesot	Vietnam		59873	10384,21	Cash on Del	Stephanie B	TGT84002	COMPLETED	Order deliv	2024-07-26	2024-09-10 23:50:33
2	45	2024-02-06	INV760912	83984 Moo West Kevin	North Dako	Djibouti		10845	884,96	Cash on Del	Gary Delgac	TEQI96332	AWAITING	Order #INV	2024-02-06	2024-03-04 20:12:42
3	103	2024-03-21	INV534662	0724 Elliott Ronald	mou Kentucky	Portugal		78847	4049,1	Buy Now Pa	Maurice Val	TNFF84472	ON_HOLD	Please cont	2024-03-21	2024-03-23 19:23:58
4	71	2023-11-23	INV830455	2631 Bento Bradford	for Arizona	Barbados		47876	145,54	Cash on Del	Connie Sma	RUFW9191	SHIPPED	Order shipp	2023-11-23	2023-12-24 13:16:06
5	177	2023-07-26	INV670692	6996 James Jennifer	shir Iowa	Cuba		10159	5027,2	Credit Card	Kenneth Mi	DVE046726	ON_HOLD	Order #INV	2023-07-26	2023-09-14 08:03:24
6	126	2023-12-15	INV459953	4097 Mitch North	Bryar Wyoming	Honduras		71076	427,98	Credit Card	Hector Hale	JZIL783451	SHIPPED	Shipment ir	2023-12-15	2024-02-09 19:30:10
7	479	2023-11-01	INV967958	573 Thomp Wilkinson	sp Iowa	Sierra Leon		14684	7984,65	Gift Card	Jeffrey Murj	YOSQ09208	ON_HOLD	Order place	2023-11-01	2023-12-27 17:02:38
8	14	2023-08-15	INV919357	58669 Hugl Logantown	Kentucky	Liechtenste		12182	1516,47	Buy Now Pa	Michael We	OC2C50465	ON_HOLD	We're waiti	2023-08-15	2023-10-05 20:53:42
9	64	2024-10-31	INV365793	72067 Matt West	Christ Tennessee	Sweden		78198	3668,51	Credit Card	Jennifer Hai	AAQL49643	ON_HOLD	Please cont	2024-10-31	2024-11-07 14:47:21
10	449	2023-09-22	INV302602	800 Sara Co North	Jenni Florida	Mexico		94469	6262,19	Cash on Del	Mrs. Jaime I	ILAG66183	AWAITING	We've recei	2023-09-22	2023-10-28 11:30:31
11	465	2024-11-23	INV692876	40226 Hop New	John Ohio	Ukraine		82428	805,11	Buy Now Pa	Jose Lloyd	YEPV40411	AWAITING	Shipping shi	2024-11-23	2025-01-02 05:51:48
12	339	2023-11-05	INV942598	214 Weiss F South	Shaw Indiana	Ecuador		52573	9304,73	Bank Transf	Ariana Hans	EKW344401	COMPLETED	Thank you f	2023-11-05	2023-12-29 15:46:56
13	422	2025-03-21	INV982098	97120 Men South	Trevc Montana	Niger		37230	6399,61	Bank Transf	Lorraine Ne	OKMP9086	ON_HOLD	Additional	2025-03-21	2025-05-02 21:25:29
14	47	2025-06-07	INV970676	895 Banks V Thomas	viev Tennessee	Spain		73008	5314,47	Buy Now Pa	Kathleen Lo	BFDA13548	AWAITING	Shipping lai	2025-06-07	2025-07-20 14:49:52
15	178	2023-11-22	INV348193	78568 Dam James	mout Louisiana	Ghana		90044	1043,34	Credit Card	Dr. Andrew	NABR5954f	SHIPPED	Order #INV	2023-11-22	2023-12-06 23:29:44
16	415	2023-12-04	INV855691	876 Wilson Robert	berg Washington	United Stat		32700	4249,25	Bank Transf	Kristen You	KYGZ56624	AWAITING	Shipping lai	2023-12-04	2024-01-31 03:45:17
17	311	2025-03-04	INV810946	46339 Coré East	Joshua Nebraska	Lebanon		54386	404,16	Credit Card	Russell Blac	VIDH46953	ON_HOLD	Please cont	2025-03-04	2025-03-20 18:48:09
18	217	2024-05-04	INV370886	286 Angela East	Ansior Arkansas	Ethiopia		26663	6285,5	Bank Transf	Brandon W	VYKU31545	AWAITING	Your order f	2024-05-04	2024-05-16 13:09:16
19	183	2023-11-1C	INV374946	750 Charle New	Hollyst Montana	Belarus		69530	7457,16	Cash on Del	Christophe	OCGJ47923	COMPLETED	Thank you f	2023-11-1C	2024-01-02 04:53:27
20	333	2023-08-15	INV239609	27234 Davi East	Nicole New Hamps	Central Afri		45326	9634,05	Gift Card	Amanda Cal	JINC35996f	ON_HOLD	Please cont	2023-08-15	2023-08-27 20:44:11
21	380	2025-06-13	INV497658	17253 Eric East	Donald Montana	Christmas Is		52660	6091,02	Credit Card	Linda Denni	JMND8068	AWAITING	Order recei	2025-06-13	2025-06-16 07:47:49
22	336	2024-06-2C	INV819325	515 Hardin Dillon	bury Louisiana	Cape Verde		97358	3383,23	Cash on Del	Jamie How	DQTY99205	SHIPPED	Order shipp	2024-06-2C	2024-07-22 19:20:34
23	38	2025-02-15	INV973372	411 Christo Dannym	out Massachus	British Indi		85495	9527,03	Bank Transf	Christophe	HAOK5040f	SHIPPED	Order shipp	2025-02-15	2025-03-11 09:34:49
24	188	2023-12-31	INV924797	697 Jordan Morgans	ide Utah	Saint Marti		86692	2183,68	Buy Now Pa	Rebecca Bee	JRYM99473	AWAITING	Order #INV	2023-12-31	2024-01-26 10:18:45
25	265	2023-12-2C	INV91437	87240 Chri Steven	stad Virginia	Namibia		60452	2508,64	Credit Card	Raymond Li	OZRS3960	AWAITING	Shipping lai	2023-12-2C	2024-02-08 09:58:42
26	26	2023-09-27	INV925140	9558 Sheltc Marqu	ezmc Texas	Madagascar		88467	10257,25	Gift Card	Gabriel Mill	QDAZ1205f	ON_HOLD	Order place	2023-09-27	2023-10-20 13:22:06
27	351	2024-09-12	INV673968	9210 Hump Hic	ksborou Oklahoma	Ireland		60863	6037,34	Gift Card	Brett Hood	QKGU4609	COMPLETED	Thank you f	2024-09-12	2024-09-13 21:35:44
28	210	2023-10-22	INV746407	46973 Sam Joel	ton New Hamps	Grenada		35324	1400,9	Cash on Del	Todd Alvara	QFSK86567	COMPLETED	Order deliv	2023-10-22	2023-12-02 16:29:04
29	459	2024-05-31	INV657596	1639 Browi Brittan	ystat Pennsylvan	Gibraltar		61051	1114,08	Credit Card	Dean Gome	YMGW134f	ON_HOLD	Order #INV	2024-05-31	2024-07-24 22:16:41
30	180	2023-08-02	INV432072	131 Jimmy North	Kayla Kansas	Bouvet Islar		96279	4220,63	Bank Transf	Christine K	SKZRS3265	ON_HOLD	We're waiti	2023-08-02	2023-09-08 19:55:10
31	194	2024-01-26	INV246264	2164 Duart George	burg Delaware	United Stat		45368	3212,09	Credit Card	Jonathan P	VQV4185f	SHIPPED	Your packag	2024-01-26	2024-01-30 07:33:41
32	458	2023-12-2C	INV98866	605 Melissa Cynthi	alanc Idaho	United Stat		88458	10763,1	Gift Card	Hannah Wri	BKWV87821	SHIPPED	Your packag	2023-12-2C	2024-02-06 17:49:56
33	15	2023-09-16	INV280120	1100 Timot Sandra	shire Montana	United Stat		57800	9665,54	Gift Card	Ricardo Lev	GKZRS5807	COMPLETED	Order deliv	2023-09-16	2023-10-02 00:14:23
34	86	2024-10-15	INV54160	80112 Rive North	Steph Indiana	Equatorial G		70227	4623,84	Bank Transf	Christina Fl	XILO2274f	AWAITING	Your order i	2024-10-15	2024-11-24 10:05:32
35	261	2024-01-12	INV264089	91852 Fran Christi	anbu New Hamps	Montserrat		66695	3400,47	Gift Card	Courtney P	HEYV51426	SHIPPED	Shipment ir	2024-01-12	2024-01-14 03:57:55
36	499	2025-01-14	INV859716	70357 Mad Frost	fort Tennessee	Hungary		38470	2906,99	Buy Now Pa	Jonathan G	EKIA90495f	AWAITING	Your order i	2025-01-14	2025-02-04 05:38:28
37	281	2025-04-12	INV468812	67955 Cars Patric	kshir Connecticut	Macao		50045	6682,55	Gift Card	Sarah Ruiz	PVPO8844f	AWAITING	Order is sch	2025-04-12	2025-05-01 00:25:20
38	140	2024-12-05	INV133977	464 Kidd Vi Juan	borou Maryland	Turks and C		65778	3821,65	Buy Now Pa	Timothy Ca	AGEM4221	SHIPPED	Shipment ir	2024-12-05	2025-01-27 04:40:19

Figure 3: invoices data sample

6.2 Invoice Items Table Sample

id	invoice_id	product_id	unit_price	quantity	created_at	updated_at
1	304	81	119,47	5	2025-03-25	2025-03-26 00:57:03
2	413	499	616,27	5	2024-06-25	2024-07-14 03:20:21
3	1	172	75,39	5	2023-06-23	2023-06-23 03:31:37
4	429	104	390,41	1	2024-08-14	2024-08-14 13:39:27
5	6	423	80,39	1	2025-01-04	2025-01-04 23:50:22
6	243	506	490,32	9	2024-06-22	2024-06-22 18:56:04
7	424	558	593,94	9	2024-06-14	2024-06-14 08:37:41
8	110	323	74,66	5	2023-11-04	2023-11-04 18:04:33
9	117	569	598,67	2	2024-12-22	2024-12-22 07:39:27
10	397	35	135,25	10	2025-01-14	2025-02-11 05:49:59
11	100	46	478,41	6	2023-09-13	2023-10-13 18:31:30
12	59	162	580,64	3	2024-01-1C	2024-01-13 18:55:49
13	148	324	689,41	4	2025-04-2C	2025-04-20 10:01:20
14	28	348	197,39	6	2024-02-12	2024-02-12 22:54:31
15	105	494	297,57	5	2025-05-3C	2025-05-30 21:29:21

Figure 4: invoice items data sample

6.3 Sample Invoice-Item Relationships

Invoice INV808802 (User 209, Total: \$10,384.21) contains:

Product 172: $\$75.39 \times 5 = \376.95

Product 256: $\$890.50 \times 3 = \$2,671.50$

Product 445: $\$1,200.75 \times 2 = \$2,401.50$

Product 521: $\$1,133.84 \times 4 = \$4,535.36$

Total calculated: \$9,985.31 (\$10,384.21 with tax/shipping)

Invoice INV760912 (User 45, Total: \$884.96) contains:

Product 499: $\$616.27 \times 1 = \616.27

Product 203: $\$134.34 \times 2 = \268.68

Total calculated: \$884.95 (\$884.96 with rounding)

7 Code Architecture and Implementation

7.1 invoices.py Key Components

```
# Core data generation components:

# 1. Payment method definitions
PAYMENT_METHODS = [
    'Bank Transfer',
    'Cash on Delivery',
    'Credit Card',
    'Buy Now Pay Later',
    'Gift Card'
]

# 2. Status workflow definitions
STATUS_CHOICES = [
    'AWAITING_FULFILLMENT',
    'ON_HOLD',
    'AWAITING_SHIPMENT',
    'SHIPPED',
    'COMPLETED'
]

# 3. Invoice generation with geographic data
user_id = random.randint(1, NUM_USERS)
invoice_date = fake.date_time_between(start_date='-2y', end_date='now')
invoice_number = f"INV{random.randint(100000,999999)}"
billing_address = fake.street_address()
billing_city = fake.city()
billing_state = fake.state()
billing_country = fake.country()
billing_postcode = fake.postcode()

# 4. Payment account generation
payment_method = random.choice(PAYMENT_METHODS)
payment_account_name = fake.name()
payment_account_number = fake.bothify(text='????#####')

# 5. Status and total calculation
status = random.choice(STATUS_CHOICES)
# Total calculated from invoice items sum
```

7.2 invoice_items_generator.py Key Components

```
# Core data generation components:

# 1. Product data integration
try:
```

```

products_df = pd.read_csv('products.csv')
products_available = True
product_ids = products_df['id'].tolist()
product_prices = dict(zip(products_df['id'], products_df['price']))
except FileNotFoundError:
    # Fallback to generated product data
    product_ids = list(range(1, 601))
    product_prices = {pid: round(random.uniform(10.0, 1000.0), 2)
                      for pid in product_ids}

# 2. Items per invoice distribution
num_items = random.choices(
    population=[1, 2, 3, 4, 5, 6, 7, 8],
    weights=[5, 15, 25, 25, 15, 10, 3, 2],
    k=1
)[0]

# 3. Quantity distribution logic
quantity = random.choices(
    [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    weights=[30, 25, 20, 10, 8, 3, 2, 1, 0.5, 0.5]
)[0]

# 4. Invoice total calculation
total_for_invoice += unit_price * quantity

# 5. Temporal relationship management
item_created_at = invoice_date + timedelta(
    minutes=random.randint(1, 60)
)
item_updated_at = item_created_at + timedelta(
    hours=random.randint(0, 48)
)

```

8 Self-Evaluation (Data Generation)

Criteria	Self-Evaluation	Notes
2 Tables Selection	1.0 / 1.0	Selected two key tables: <code>invoices</code> and <code>invoice_items</code> with proper relationship.
Sample Data	2.0 / 2.0	All data is meaningful, realistic, and aligned with e-commerce invoice context.
Data Generation Report	1.0 / 1.0	Report includes comprehensive field rules, tools used, implementation details, and samples.